Security Audit Report

Oracle Aggregator

Date	December 26, 2025
Project	Oracle Aggregator
Version	1.0

Contents

Disclosure 1
Disclaimer 2
Scope 3
Assessment Overview 4
Assessment Components 5
Executive Summary 6
Code Base 8
Severity Classification 9
Finding Severity Ratings 10
Findings 11
[H 401] Minswap Price Mismatch 12
[M 301] Unverified Liqwid NFT Name 15
[M 302] Incorrect Divide By Zero Price Handling 17
[I 101] Redundant Oracle Source Type tSplashLiquidityPoolCpammPolicyId 19

Disclosure

This document contains proprietary information belonging to No Witness Labs. Duplication, redistribution, or use, in whole or in part, in any form, requires prior written consent from No Witness Labs.

Nonetheless, both the customer **Danogo** and No Witness Labs are authorized to share this document with the public to demonstrate security compliance and transparency regarding the outcomes of the Protocol.

Disclaimer

This review reflects the state of the codebase at the time of assessment and should be regarded as a point-in-time evaluation. Any changes made after the assessment period fall outside the scope of this report.

Although every effort was made to identify potential risks and vulnerabilities, no security review can guarantee the discovery of all issues. The information and recommendations in this report are provided solely for technical purposes and must not be considered investment advice.

To strengthen security assurance, projects are encouraged to seek multiple independent reviews and to supplement audits with practices such as bug bounty programs.

The scope of this audit was limited to the code provided. It did not extend to compiler-level artifacts (e.g., generated UPLC code) or to areas beyond the explicitly reviewed codebase. Additional testing methodologies, including unit testing and property-based testing, were also outside the scope of this engagement.

Scope

In scope

- On-chain validators and supporting logic at the time of review.
- Datum/redeemer structures and on-chain invariants (value, units/rates, cross-phase checks).
- Use of reference UTxOs for configuration and script parameters.

Out of scope

- Compiler layer / UPLC output and toolchain internals.
- Creation of new unit tests or property-based tests.
- Off-chain libraries, front-end apps, wallet UX (incl. blind-signing), unless explicitly constrained on-chain.
- Third-party protocol internals (bridges/liquidity venues), centralized services, exchanges, oracles, external price feeds.
- Node operations, networking, mempool policy variance, ordering/MEV-like effects.

Assessment Overview

From **November 18, 2025** to **December 24, 2025**, **Danogo** engaged No Witness Labs to conduct a comprehensive security assessment of the **Oracle Aggregator** protocol. The evaluation was performed in accordance with established industry best practices for code auditing and protocol analysis.

The audit followed a structured five-phase process:

- **Planning** – Defined the assessment scope and objectives, and aligned expectations with the client.
- **Discovery** – Acquired an understanding of the protocol through client discussions, documentation review, and preliminary analysis of architecture and design.
- **Manual Revision** – Conducted an in-depth review of the codebase to identify potential vulnerabilities, design weaknesses, and security risks.
- **Validation** – Verified client-provided fixes and obtained acknowledgments for resolved issues.
- **Reporting** – Documented all confirmed findings, assigned severity levels, and provided actionable recommendations for remediation.

The team also evaluated protocol efficiency and potential optimizations. Findings were prioritized by severity level.

Assessment Components

Manual revision

Our manual code auditing is focused on a wide range of attack vectors, including but not limited to:

- UTXO Value Size Spam (Token Dust Attack)
- Large Datum or Unbounded Protocol Datum
- EUTXO Concurrency DoS
- Unauthorized Data modification
- Multisig PK Attack
- Infinite Mint
- Incorrect Parameterized Scripts
- Other Redeemer
- Other Token Name
- Arbitrary UTXO Datum
- Unbounded protocol value
- Foreign UTXO tokens
- Double or Multiple satisfaction
- Locked Ada
- Locked non Ada values
- Missing UTXO authentication
- UTXO contention

Executive Summary

Project Overview

Oracle Aggregator is a centralized on-chain aggregator of oracle price feeds. It enables on-chain derivation of customized price feeds using available oracle feeds. There are no limits to the number of oracle sources it can work with while easily incorporating new sources. Each final price can be derived by averaging out multiple independent price feeds from various sources and subsequently undergoing maximum allowed deviation checks. This mechanism results in protection from suspicious input price which diverges more than the allowed threshold from final average price.

The aggregator relies on two different types of reference inputs authenticated by Global Config NFT and Oracle Path FTs to obtain `OracleGlobalConfig` and `OraclePathDatum`s respectively.

- `OracleGlobalConfig` serves as an authorized source of oracle identifiers belonging to Liqwid, Orcfax and Minswap protocols.
- `OraclePathDatum` contains token pair specific information like price derivation paths, oracle source identifiers and price deviation threshold.

It is crucial to note that manipulating these datum values can result in incorrect price calculations or utilizing fake prices resulting in financial losses. Currently, these NFTs are directly managed by the Danogo team.

Code Base

Repository

https://github.com/danogo-finance/oracle-aggregator

Commit

`9352e50b11699cd7491a90fa53f396499f9e408c`

Files Audited

Files	SHA256 Checksum
validators/oracle.ak	7e581fa8c05846fe364043edcdd77be14fb0a2f847c53317dabd7ae504cb91b9
lib/constants.ak	30a5bcd23995ea6ee3d2861eba22638c5e31320b03f47829b283b84fbd561a21
lib/helpers.ak	130564616bd269292ef198a8cd93d8f14193c63d1839968a80ec93735648bcaa
lib/liqwid_types.ak	96e615bb8e18e300e6864759adb3d34c1e548b418e74f70a3eed2734028fbf6a
lib/oracle_types.ak	d140bec8d4c116b1e0760860ce5a323bff060243851a28dea2cfdb1450955f2a
lib/sc_fallback.ak	16fa72b18a90d45dbb3f7514cd2669c49a0c5b33ea9205d04830b333439b169e
lib/sc_oracle.ak	d626ec0c396aff85d3bd6347667b8b11718d8c0830153b25f27c06bcb9901a99
lib/types.ak	72ecb98e06fd99e4583fb7a76070e54131a6f77eddb2078f010485f06f9c8b33
lib/utils.ak	6be14f3835af2cfea20a81ccc76988dc87d82558e7fafb6133fea688c9659aad

Severity Classification

- **Critical**: Vulnerabilities that enable direct theft of assets, permanent loss of funds, or catastrophic protocol failure. These represent immediate and severe financial risk with straightforward exploitation paths.
- **High**: Vulnerabilities that can result in financial loss or significant harm to users or the protocol, though impact may be limited to specific functions or require certain conditions. Exploitation paths exist but may be more complex than Critical Vulnerabilities.
- **Medium**: Vulnerabilities that do not directly result in financial loss but may temporarily impair functionality or create exploitable conditions. These are typically limited to state manipulations and include issues such as front-running susceptibility or logic flaws that compromise transaction integrity.
- **Low**: Low-impact Vulnerabilities with limited attack vectors, minimal attacker incentive, or requiring highly specific scenarios. These typically do not result in financial loss or significant harm to users or the protocol.
- **Informational**: Findings without immediate security or financial implications. These include optimization opportunities, code quality improvements, inconsistencies in naming conventions, design suggestions, and gaps in documentation.

Impact–Likelihood Severity Matrix

	High likelihood	Medium likelihood	Low likelihood
High impact	C	H / M	M
Medium impact	H	M	M / L
Low impact	M	M / L	L

Finding Severity Ratings

The following table defines levels of severity and score range that are used throughout the document to assess vulnerability and risk impact

Level	Severity	Status
5	Critical	0
4	High	1
3	Medium	2
2	Low	0
1	Informational	1

Findings

ID	Title	Severity	Status
401	Minswap Price Mismatch	High	Resolved
301	Unverified Liqwid NFT Name	Medium	Resolved
302	Incorrect Divide By Zero Price Handling	Medium	Resolved
101	Redundant Oracle Source Type tSplashLiquidityPool-CpammPolicyId	Informational	Resolved

[H 401] Minswap Price Mismatch

Level	Severity	Status
4	High	Resolved

Affected File: `lib/helpers.ak`

Description

```aiken
// In lib/helpers.ak within fn handle_oracle_idx()

else if oracle_utxo_type == constants.utMinswapLP {
  let (lp_asset_name, ft_qty) =
    assets.reduce(
      value,
      (constants.empty_bytearray, 0),
      fn(pid, tn, qty, (lp_asset_name, ft_qty) as ret) {
        if pid == minswap_pool_authen_asset_pid {
          if tn == minswap_pool_authen_asset_name {
            (lp_asset_name, qty)
          } else {
            (tn, ft_qty) // <-- Lexicographically later lp_asset_name will be selected
          }
        } else {
          ret
        }
      },
    )
  expect ft_qty == 1
  //
  expect MinswapPoolDatum { total_liquidity, reserve_a, reserve_b, .. }: MinswapPoolDatum =
    datum
  up_s9(
    [
      Pair(
        lp_asset_name,
        LiquidityPoolCpammData {
          ...
        },
      ),
      ..s9
    ],
```

```
  )
}
```

Ideally, a Minswap Liquidity Pool UTxO should contain only one LP asset token belonging to pool's token pair e.g. ADA-SNEK Pool should only have LP_ADA_SNEK LP token in it. However, if it contains other LP tokens in a pool (ADA-SNEK pool containing LP_ADA_SNEK and LP_ADA_WMT LP tokens), current Oracle implementation allows using ADA-SNEK pool prices for ADA-WMT pool should LP_ADA_WMT token name be lexicographically later than LP_ADA_SNEK. This happens because `lp_asset_name` is replaced with a lexicographically later token name even when a token name was found, which happens when multiple LP tokens are present in a single liquidity pool UTxO.

While Minswap may enforce one LP token per pool UTxO, an external protocol's invariant can never be safely relied upon unless absolutely necessary.

Recommendation

We recommend to fail validation when another LP token is found in a pool UTXO.

```
else if oracle_utxo_type == constants.utMinswapLP {
  expect (Some(lp_asset_name), ft_qty) =
    assets.reduce(
      value,
      (None, 0),
      fn(pid, tn, qty, (lp_asset_name, ft_qty) as ret) {
        if pid == minswap_pool_authen_asset_pid {
          if tn == minswap_pool_authen_asset_name {
            (lp_asset_name, qty)
          } else {
            expect lp_asset_name == None // <-- validation failure in case of multiple LP tokens
            (tn, ft_qty)
          }
        } else {
          ret
        }
      },
    )
  expect ft_qty == 1

  ...

```

```
}
```

Resolution

Resolved in commit hash `6063867ae88c313af5668006e9b6c57d167f8836`

[M 301] Unverified Liqwid NFT Name

Level	Severity	Status
3	Medium	Resolved

Affected File: `lib/helpers.ak`

Description

```
else if oracle_utxo_type == constants.utLiqwidMarketState {
  let pid, _ <- get_nft_from_value(value) // <-- unverfied asset name
  expect (_, _, _, _, _, _, interest_rate, _, _, qtoken_rate, _): LiqwidMarketStateDatum =
    datum
  up_s2([Pair(pid, (interest_rate, qtoken_rate)), ..s2])
} else if oracle_utxo_type == constants.utLiqwidMarketParam {
  let pid, _ <- get_nft_from_value(value) // <-- unverfied asset name
  ...
  up_s3([Pair(pid, compound_rate), ..s3])
}
```

UTxOs bearing Liqwid Market State and Market Param datums are identified by protocol specific NFTs. These NFTs have empty bytearray as their asset name. As a consequence, validators omit checking asset name while verifying these NFTs. This results in incomplete verfication and can cause misidentification with other Liqwid NFTs having the same policy id with a different asset name. Essentially, downgrading NFT based authentication to FT based authentication, allowing usage of fake price feeds in the downstream protocol; subject to ease of availability and control of such Liqwid FTs.

Recommendation

To accurately verify asset name of Liqwid NFTs belonging to Market State and Param UTxOs.

```
else if oracle_utxo_type == constants.utLiqwidMarketState {
  let pid, name <- get_nft_from_value(value)
  expect name == #"" // <-- verfied asset name
  expect (_, _, _, _, _, _, interest_rate, _, _, qtoken_rate, _): LiqwidMarketStateDatum =
    datum
  up_s2([Pair(pid, (interest_rate, qtoken_rate)), ..s2])
} else if oracle_utxo_type == constants.utLiqwidMarketParam {
  let pid, name <- get_nft_from_value(value)
  expect name == #"" // <-- verfied asset name
  ...
  up_s3([Pair(pid, compound_rate), ..s3])
```

```
}
```

Resolution

Resolved in commit hash `6063867ae88c313af5668006e9b6c57d167f8836`

[M 302] Incorrect Divide By Zero Price Handling

Level	Severity	Status
3	Medium	Resolved

Affected File: `lib/helpers.ak`

Description

```
pub fn calc_out_amount(
  in_amount: Int,
  numerator: Int,
  denominator: Int,
  is_reverse: Bool,
) -> Int {
  if numerator == 0 || denominator == 0 { // <-- zero denominator resulting in zero price
    0
  } else if is_reverse {
    in_amount * denominator / numerator
  } else {
    in_amount * numerator / denominator
  }
}
```

`calc_out_amount` implementation incorrectly handles divide by zero error condition by reducing the price to zero should either numerator or denominator be zero without taking `is_reverse` into account. The price should only be zero when the actual numerator (`denominator` if `is_reverse` is true else `numerator`) is zero. Should the actual price denominator be zero, then corresponding output token amount is infinite (1/0) which is incorrectly simplified to being zero currently. As a result this can reduce a valuable token's price to zero simply because it's price path config used a token's price feed for price derivation and that said token's price has now reduced to zero.

To illustrate this with an example, let's take conversion of token A to token B. A -> B Price Paths

1. A -> X, X -> Y, Y -> B
2. A -> Z, Z -> B

Price of token Y has gone to zero so `price_y_to_x = numerator / denominator` where `denominator == 0` in reality tends to infinity, but it is reduced to zero as per current implementation. This makes the output amount of token B as zero i.e. `price_a_to_b == 0`. Resulting in a critical price calculation error, reducing every token's price to zero whose price path has a zero value token or the price feed is incorrectly reported as zero by an oracle.

Pre-requisites For Severe Financial Losses

1. A token used in a price path, whose actual price goes to zero or a faulty oracle feed reporting it's price as zero.
2. A token pair only having a single affected price path or multiple price paths which are all affected due to inclusion of zero value tokens.
3. Protocol's price monitoring off-chain fails to observe zero price anomalies and therefore does not update price path configurations in time.

Recommendation

To fail validation when actual denominator is zero. It is also recommended to have off-chain price monitoring systems in place which notifies protocol team whenever an oracle price feed becomes zero, to swiftly evaluate it's impact on price paths and update them if needed.

```
pub fn calc_out_amount(
  in_amount: Int,
  numerator: Int,
  denominator: Int,
  is_reverse: Bool,
) -> Int {
  if is_reverse {
    expect numerator > 0
    in_amount * denominator / numerator
  } else {
    expect denominator > 0
    in_amount * numerator / denominator
  }
}
```

Resolution

Resolved in commit hash `c24ed71066590d4deb8ab31dded056b012d1c70e`

[I 101] Redundant Oracle Source Type tSplashLiquidityPool-CpammPolicyId

Level	Severity	Status
1	Informational	Resolved

Affected File: `lib/sc_oracle.ak`

Description

Oracle Source Type `tSplashLiquidityPoolCpammPolicyId` obtains oracle feed information from `splash_pools_cpamm: Pairs<ByteArray, LiquidityPoolCpammData>` identical to Oracle Source Type `tSplashLiquidityPoolCpamm`. `tSplashLiquidityPoolCpammPolicyId` implementation obtains the Splash NFT indirectly by sourcing policy id from GlobalConfig and asset name from `oracle_source_info`. Such distinct handling is not required by these NFTs and can be well very accomodated by `tSplashLiquidityPoolCpamm` implementation. This makes the validator implementation more concise and efficient as NFT hash is readily available through `oracle_source_info`.

Recommendation

To remove `tSplashLiquidityPoolCpammPolicyId` Oracle Source Type and use `tSplashLiquidityPoolCpamm` instead for all the required Splash CPAMM feeds.

Resolution

Resolved in commit hash `6063867ae88c313af5668006e9b6c57d167f8836`

End of Report

Copyright © 2025